

计算器--多态实现

复习

- | | |
|----------------|----|
| 1. 多态的基本概念 | 了解 |
| 2. 构成多态的条件 | 重点 |
| 3. 引用实现多态 | 熟悉 |
| 4. 虚函数 | 重点 |
| 5. 虚析构 | 重点 |
| 6. 虚函数表 | 重点 |
| 7. C++如何实现动态绑定 | 熟悉 |
| 8. 抽象基类和纯虚函数 | 重点 |

目录

1. 计算器概述
2. 设计基类
3. 设计子类
4. 设计不同计算方法
5. 现实主函数
6. 改成多文件形式

计算器概述

使用多态实现计算器的加减乘除等计算操作



案例演示

菜单函数

```
void menu()
{
    cout << "*****" << endl
        << "***** 1.加法 *****" << endl
        << "***** 2.减法 *****" << endl
        << "***** 3.乘法 *****" << endl
        << "***** 4.除法 *****" << endl
        << "***** 5.取余 *****" << endl
        << "***** 6.乘方 *****" << endl
        << "***** 7.开根 *****" << endl
        << "***** 8.次幂 *****" << endl
        << "***** Q.退出 *****" << endl
        << "*****" << endl;
}
```

设计基类

实现个基类，利用多态的知识，方便拓展与维护！

//抽象计算基类

```
class AbstractCalculator {
```

```
public:
```

```
    //写上virtual关键字，方便下面的类继承时的同名函数进行重写,纯虚函数，用来得到计算结果
```

```
    virtual double getResult() = 0;
```

```
    double num1, num2;
```

```
};
```

设计子类

子类继承继承基类。

```
class AddCalculator : public AbstractCalculator { //加法类
```

```
public:
```

```
    double getResult() {  
        return num1 + num2;  
    }
```

```
};
```

```
class SubCalculator : public AbstractCalculator {
```

```
public:
```

```
    double getResult() {  
        return num1 - num2;  
    }
```

```
};
```

```
class MulCalculator : public AbstractCalculator {
```

```
public:
```

```
    double getResult() {  
        return num1 * num2;  
    }
```

```
};
```

设计子类

```
class DivCalculator : public AbstractCalculator //division 除法
{
public:
    double getResult()
    {
        return num1 / num2;
    }
};
```

```
class SurCalculator : public AbstractCalculator //Surplus 取余
{
public:
    double getResult()
    {
        return (int)num1 % (int)num2;
    }
};
```

```
class PowCalculator : public AbstractCalculator //Power 乘方
{
public:
    double getResult()
    {
        return num1 * num1;
    }
};
```

设计子类

```
class SqrtCalculator : public AbstractCalculator //Sqrt 开根
```

```
{  
public:  
    double getResult()  
    {  
        return sqrt(num1);  
    }  
};
```

```
class PowpCalculator : public AbstractCalculator //Powp 次幂
```

```
{  
public:  
    double getResult()  
    {  
        return pow(num1, num2); //计算num1的num2次方  
    }  
};
```

设计不同计算方法

```
void Add(int a, int b) {  
    cout << "请输入a和b的值" << endl;  
    cin >> a >> b;  
    AbstractCalculator* p = new AddCalculator;  
    p->num1 = a;  
    p->num2 = b;  
    cout << p->num1 << "+" << p->num2 << " = " << p->getResult() << endl;  
    delete p;  
    cout << "按任意键继续" << endl;  
    system("pause");  
    system("cls");  
}
```

设计不同计算方法

```
void Pow1(int a, int b) {
    cout << "请输入a和b的值" << endl;
    cin >> a >> b;
    AbstractCalculator* p = new PowpCalculator;
    p->num1 = a;
    p->num2 = b;
    cout << "result:" << p->getResult() << endl;
    delete p;
    cout << "按任意键继续" << endl;
    system("pause");
    system("cls");
}

void Sqrt1(int a) {
    cout << "请输入a的值" << endl;
    cin >> a;
    AbstractCalculator* p = new SqrtCalculator;
    p->num1 = a;
    cout << "result:" << p->getResult() << endl;
    delete p;
    cout << "按任意键继续" << endl;
    system("pause");
    system("cls");
}

void Pow(int a) {
    cout << "请输入a的值" << endl;
    cin >> a;
    AbstractCalculator* p = new PowCalculator;
    p->num1 = a;
    cout << "result:" << p->getResult() << endl;
    delete p;
    cout << "按任意键继续" << endl;
    system("pause");
    system("cls");
}
```

设计不同计算方法

```
void Sur(int a, int b) {
    cout << "请输入a和b的值" << endl;
    cin >> a >> b;
    AbstractCalculator* p = new SurCalculator;
    p->num1 = a;
    p->num2 = b;
    cout << "result:" << p->getResult() << endl;
    delete p;
    cout << "按任意键继续" << endl;
    system("pause");
    system("cls");
}

void Div(int a, int b) {
    cout << "请输入a和b的值" << endl;
    cin >> a >> b;
    AbstractCalculator* p = new DivCalculator;
    p->num1 = a;
    p->num2 = b;
    cout << "result:" << p->getResult() << endl;
    delete p;
    cout << "按任意键继续" << endl;
    system("pause");
    system("cls");
}
```

设计不同计算方法

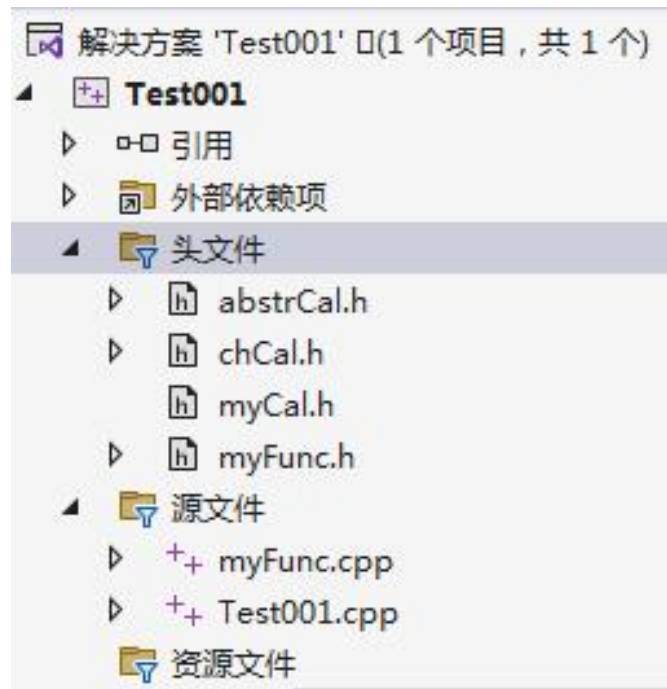
```
void Mul(int a, int b) {
    cout << "请输入a和b的值" << endl;
    cin >> a >> b;
    AbstractCalculator* p = new MulCalculator;
    p->num1 = a;
    p->num2 = b;
    cout << "result:" << p->getResult() << endl;
    delete p;
    cout << "按任意键继续" << endl;
    system("pause");
    system("cls");
}

void Sub(int a, int b) {
    cout << "请输入a和b的值" << endl;
    cin >> a >> b;
    AbstractCalculator* p = new SubCalculator;
    p->num1 = a;
    p->num2 = b;
    cout << "result:" << p->getResult() << endl;
    delete p;
    cout << "按任意键继续" << endl;
    system("pause");
    system("cls");
}
```

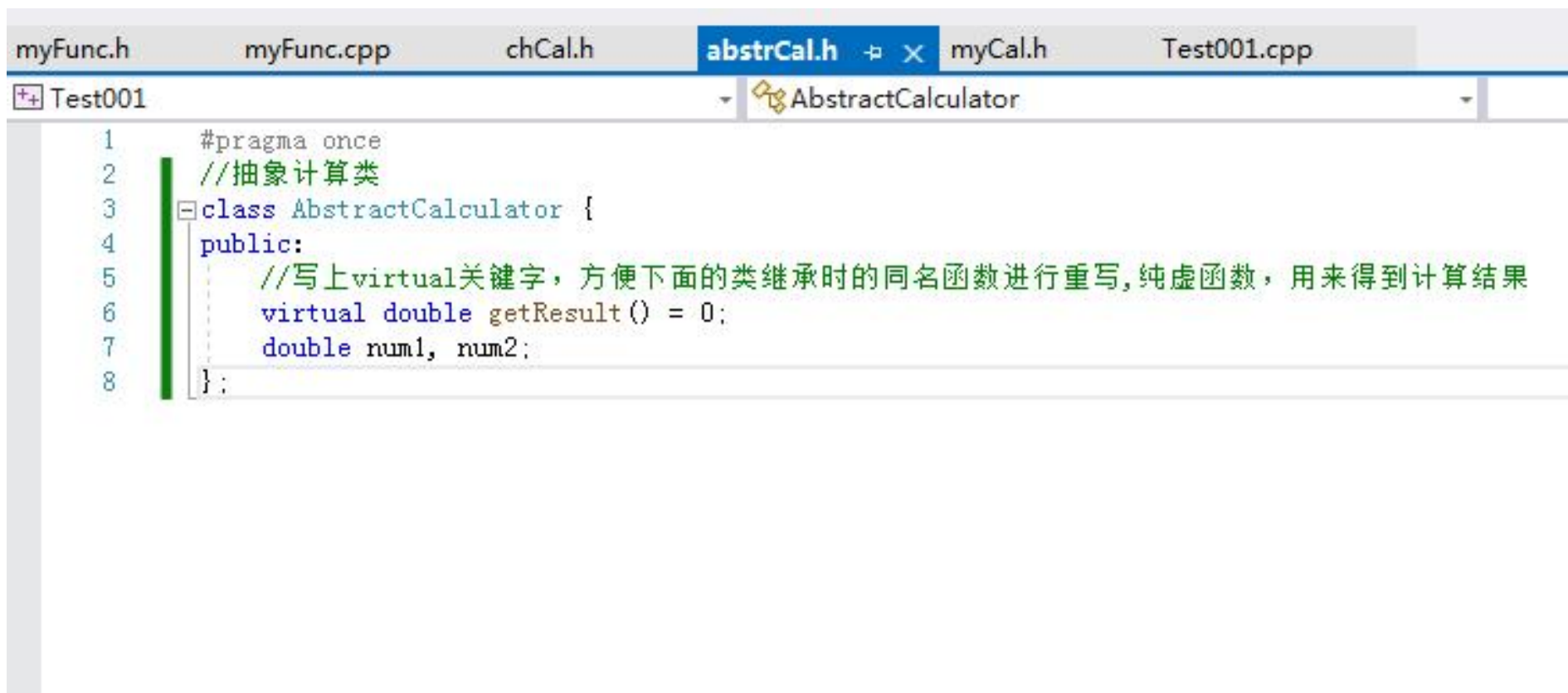
实现主函数

```
int main() {  
  
    char select;  
    double a = 0, b = 0;  
    while (1)  
    {  
        menu();  
        cout << "请选择序号进行计算 : ";  
        cin >> select;  
        switch (select)  
        {  
            case '1':  
                Add(a, b); //加法  
                break;  
            case '2':  
                Sub(a, b); //减法  
                break;  
            case '3':  
                Mul(a, b); //乘法  
                break;  
            case '4':  
                Div(a, b); //除法  
                break;  
            case '5':  
                Sur(a, b); //取余  
                break;  
            case '6':  
                Pow(a); //乘方  
                break;  
            case '7':  
                Sqrt1(a); //开根  
                break;  
            case '8':  
                Pow1(a, b); //次幂  
                break;  
            case 'Q':  
                cout << "确定退出吗 ? (Y/N)" << endl;  
                cin >> select;  
                if (select == 'Y') {  
                    cout << "欢迎下次使用" << endl; //退出  
                    return 0;  
                    break;  
                }  
            default:  
                exit(0);  
                break;  
        }  
    }  
    system("pause");  
    return 0;  
}
```

改成多文件形式



改成多文件形式-基类

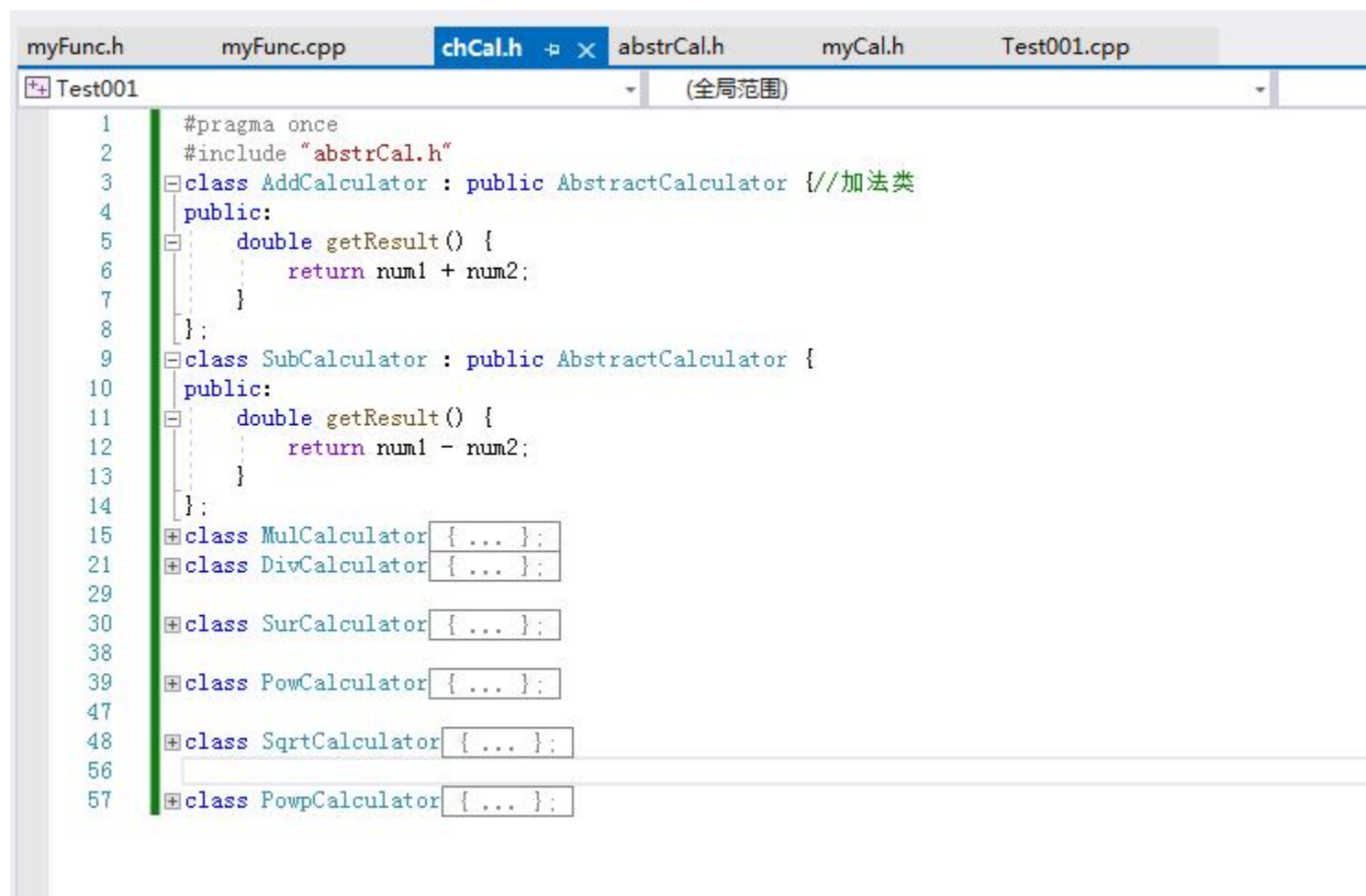


The screenshot shows a C++ IDE with several open files: myFunc.h, myFunc.cpp, chCal.h, abstrCal.h (active), myCal.h, and Test001.cpp. The active file, abstrCal.h, contains the following code:

```
1  #pragma once
2  //抽象计算类
3  class AbstractCalculator {
4  public:
5      //写上virtual关键字，方便下面的类继承时的同名函数进行重写, 纯虚函数，用来得到计算结果
6      virtual double getResult() = 0;
7      double num1, num2;
8  };
```

The code defines an abstract class `AbstractCalculator` with a public virtual function `getResult()` and two data members `num1` and `num2`. The function is marked as a pure virtual function with `= 0`.

改成多文件形式-子类



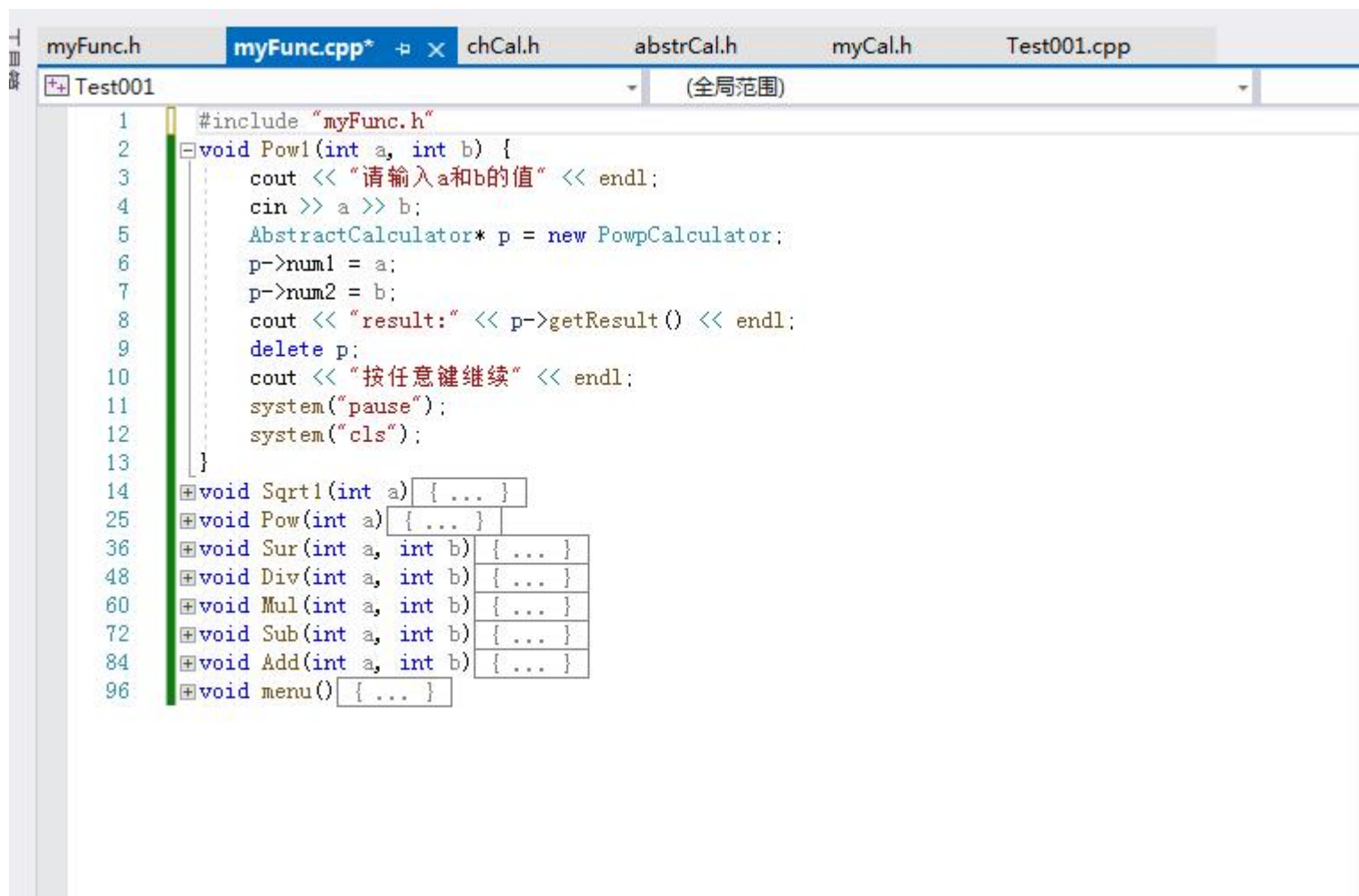
```
myFunc.h  myFunc.cpp  chCal.h  abstrCal.h  myCal.h  Test001.cpp
Test001
1  #pragma once
2  #include "abstrCal.h"
3  class AddCalculator : public AbstractCalculator { //加法类
4  public:
5      double getResult() {
6          return num1 + num2;
7      }
8  };
9  class SubCalculator : public AbstractCalculator {
10 public:
11     double getResult() {
12         return num1 - num2;
13     }
14 };
15 class MulCalculator { ... };
21 class DivCalculator { ... };
29
30 class SurCalculator { ... };
38
39 class PowCalculator { ... };
47
48 class SqrtCalculator { ... };
56
57 class PowpCalculator { ... };
```

改成多文件形式-函数头文件



```
1 #pragma once
2 #include<iostream>
3 #include "chCal.h"
4 using namespace std;
5
6 void menu();
7 void Add(int, int);
8 void Sub(int a, int b);
9 void Mul(int a, int b);
10 void Div(int a, int b);
11 void Sur(int a, int b);
12 void Pow(int a);
13 void Sqrt1(int a);
14 void Pow1(int a, int b);
```

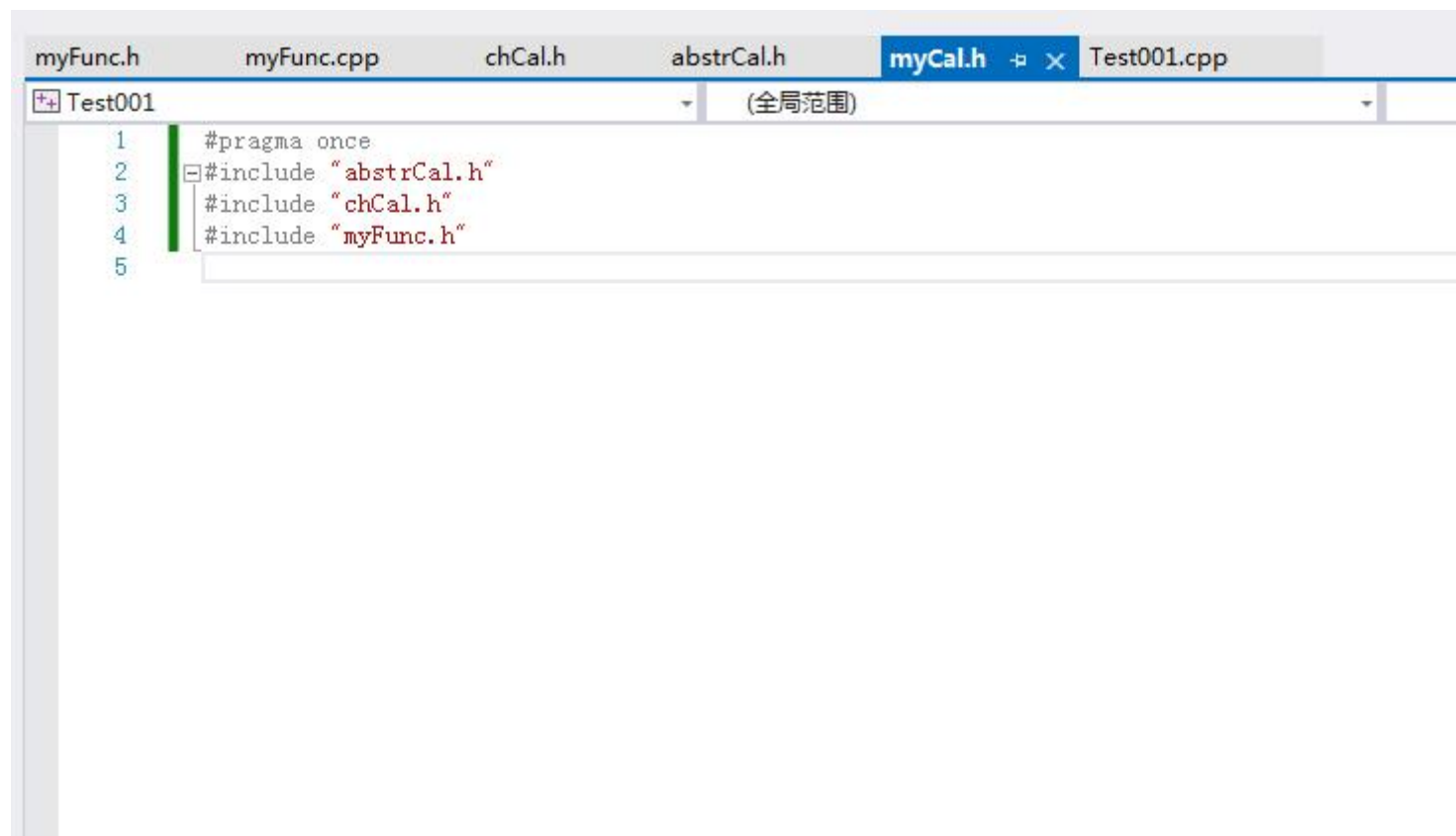
改成多文件形式-函数具体实现



The screenshot shows a C++ IDE with several files open: myFunc.h, myFunc.cpp*, chCal.h, abstrCal.h, myCal.h, and Test001.cpp. The Test001.cpp file is selected, and the code is displayed in the main editor area. The code includes the myFunc.h header and implements the Powl function, which prompts the user for input, calculates the result using a PowpCalculator object, and displays the result. Below the main function, there are several other function declarations for Sqrt1, Pow, Sur, Div, Mul, Sub, Add, and menu, each with a placeholder for its implementation.

```
1  #include "myFunc.h"
2  void Powl(int a, int b) {
3      cout << "请输入a和b的值" << endl;
4      cin >> a >> b;
5      AbstractCalculator* p = new PowpCalculator;
6      p->num1 = a;
7      p->num2 = b;
8      cout << "result:" << p->getResult() << endl;
9      delete p;
10     cout << "按任意键继续" << endl;
11     system("pause");
12     system("cls");
13 }
14 void Sqrt1(int a) { ... }
25 void Pow(int a) { ... }
36 void Sur(int a, int b) { ... }
48 void Div(int a, int b) { ... }
60 void Mul(int a, int b) { ... }
72 void Sub(int a, int b) { ... }
84 void Add(int a, int b) { ... }
96 void menu() { ... }
```

改成多文件形式



The screenshot shows a C++ IDE with a project named 'Test001'. The project structure is as follows:

- myFunc.h
- myFunc.cpp
- chCal.h
- abstrCal.h
- myCal.h (selected)
- Test001.cpp

The 'Test001' folder is expanded, showing the following code:

```
1 #pragma once
2 #include "abstrCal.h"
3 #include "chCal.h"
4 #include "myFunc.h"
5
```

知识点总结

1. 计算器概述
2. 设计基类
3. 设计子类
4. 设计不同计算方法
5. 实现主函数
6. 改成多文件形式

作业

1. 自己能够从头到尾写出计算器代码，多态实现，并且多文件编程，添加自己喜欢的运算类，增加计算机器的多样性。

下节课预告

文件操作